Kodewala Academy – Java Backend Tests

Complete Question Bank

Test 4 – Java Backend Test # 4 (31 questions)

Test 4 – Java Backend Test # 4 — 31 questions

Q1. What is the output?

```
String s1 = new String("Java");
String s2 = new String("Java");
System.out.println(s1 == s2);
```

A. true
B. false
C. 0
D. 1

Correct answer: B. false
Explanation: new always creates a distinct heap object, and == compares references.

Q2. What is the output?

```
String s1 = "Java";
String s2 = new String("java");
System.out.println(s1.equals(s2));
```

A. true
B. Compilation Error
C. false
D. 1

Correct answer: C. false
Explanation: equals() is case-sensitive: "Java" ≠ "java".

Q3. What is the output?

```
String s = "Hello";
s.concat(" World");
System.out.println(s);
```

A. Hello World
B. Compilation Error
C. Runtime Error
D. Hello

Correct answer: D. Hello
Explanation: Strings are immutable. concat returns a new String that was discarded, so s is unchanged.

Q4. Which method compares String contents?

A. ==
B. equals()
C. hashCode()
D. compare()

Correct answer: B. equals()
Explanation: equals() compares content; == compares references.

Q5. Which statement is TRUE?

A. String is mutable
B. StringBuilder is immutable
C. String is immutable
D. StringBuffer is immutable

Correct answer: C. String is immutable
Explanation: String objects cannot change after creation; StringBuilder/StringBuffer are mutable.

Q6. Which method is inherited by every Java class?

A. None
B. Both - equals() and hashCode()
C. hashCode()
D. equals()

Correct answer: B. Both - equals() and hashCode()
Explanation: Every class inherits from Object, which provides both equals() and hashCode().

Q7. What happens if equals() is not overridden?

A. Compares content
B. Uses == internally
C. Compilation Error
D. Runtime Exception

Correct answer: B. Uses == internally
Explanation: Object.equals() compares references, behaving like ==.

Q8. Which class is final?

A. Object
B. Thread
C. String
D. Exception

Correct answer: C. String
Explanation: String is declared final, so it cannot be extended.

Q9. When is a constructor called?

A. Method call
B. Object creation
C. Compilation
D. Garbage collection

Correct answer: B. Object creation
Explanation: A constructor runs when an object is created with new.

Q10. If no constructor is written:

A. Compilation Error
B. JVM creates
C. Compiler provides default constructor
D. None

Correct answer: C. Compiler provides default constructor
Explanation: The compiler adds a no-arg default constructor.

Q11. The first statement inside a constructor can be:

A. super()
B. this()
C. either super() or this()
D. anything

Correct answer: C. either super() or this()
Explanation: The first statement may be this(...) or super(...); otherwise the compiler inserts super().

Q12. Which keyword invokes the parent constructor?

A. parent()
B. this()
C. super()
D. extends()

Correct answer: C. super()
Explanation: super() calls the superclass constructor.

Q13. Object class is:

A. Parent of all classes
B. Interface
C. Final class
D. Abstract class

Correct answer: A. Parent of all classes
Explanation: java.lang.Object is the root class of every Java class.

Q14. Instance variable is created:

A. Once
B. During class loading
C. With every object
D. During compilation

Correct answer: C. With every object
Explanation: Each object gets its own copy when it is created.

Q15. A Java application receives thousands of requests simultaneously. Each request builds a large string by appending multiple values. The same string object is shared by multiple threads.

A. String
B. StringBuilder
C. StringBuffer
D. Thread

Correct answer: C. StringBuffer
Explanation: A shared mutable string needs thread safety. StringBuffer's methods are synchronized; StringBuilder is not.

Q16. Why is StringBuilder preferred over StringBuffer in this program?

```
public class Demo {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < 100000; i++) {
            sb.append(i);
        }
        System.out.println(sb.length());
    }
}
```

A. StringBuilder uses less memory
B. StringBuilder is immutable.
C. StringBuffer cannot append integers.
D. The program is single-threaded, so synchronization overhead is unnecessary.

Correct answer: D. The program is single-threaded, so synchronization overhead is unnecessary.
Explanation: Only one thread is used, so StringBuffer's synchronization is pure overhead. StringBuilder is faster here (and it's mutable, not immutable).

Q17. A developer accidentally loses the source code (.java files), but the compiled .class files are still available. Which statement is correct?

A. It is impossible to recover the Java code.
B. A Java decompiler can generate readable Java source code from the .class files, although comments and original formatting are usually lost.
C. The JVM can automatically recreate the original source code with comments.
D. The .class file contains only machine code, so decompilation is impossible.

Correct answer: B. A Java decompiler can generate readable Java source code from the .class files, although comments and original formatting are usually lost.
Explanation: Bytecode retains enough structure to be decompiled, but comments and formatting are lost.

Q18. What happens? (Employee does NOT override equals().)

```
Employee e1 = new Employee(101);
Employee e2 = new Employee(101);
System.out.println(e1.equals(e2));
```

A. true
B. false
C. Compilation Error
D. Runtime Exception

Correct answer: B. false
Explanation: The default equals() compares references; two separate objects are not equal.

Q19. A banking application updates a shared transaction log using multiple threads simultaneously. Which class should be used?

A. String
B. StringBuilder
C. StringBuffer
D. char[]

Correct answer: C. StringBuffer
Explanation: StringBuffer is synchronized and thread-safe.

Q20. What will be printed?

```
String id1 = new String("P101");
String id2 = new String("P101");
if(id1 == id2){
    System.out.println("Cache Hit");
}else{
    System.out.println("Cache Miss");
}
```

A. Cache Hit
B. Cache Miss
C. Compilation Error
D. Depends on JVM

Correct answer: B. Cache Miss
Explanation: Two new String objects are different references, so == is false. Use equals().

Q21. How many String objects are created at runtime?

```
String s = "Java" + "Backend";
```

A. 0
B. 1
C. 2
D. 3

Correct answer: B. 1
Explanation: Concatenating two literals is a compile-time constant ("JavaBackend"), so only one pooled String exists.

Q22. Why is the constructor made private?

```
public class Utility {
    private Utility()
    {
    }
}
```

A. To improve performance.
B. To prevent object creation because the class contains only utility/static methods.
C. To support inheritance.
D. To reduce memory usage.

Correct answer: B. To prevent object creation because the class contains only utility/static methods.
Explanation: A private constructor stops instantiation (as with utility classes and singletons).

Q23. Output?

```
String s1 = "Java";
String s2 = new String("Java");
String s3 = s2.intern();
System.out.println(s1 == s2);
System.out.println(s1 == s3);
System.out.println(s2 == s3);
```

A. true, true, true
B. false, true, false
C. false, false, true
D. true, false, false

Correct answer: B. false, true, false
Explanation: s2 is a heap object (≠ s1). intern() returns the pooled instance, same as s1, so s1 == s3 is true and s2 == s3 is false.

Q24. Order class has only Order(String orderId) and no equals()/hashCode(). Two orders new Order("ORD101") print "New Order" with o1.equals(o2), but the business wants same orderId to be equal. What is the best solution?

```
public class Order {
    private String orderId;
    public Order(String orderId) { this.orderId = orderId; }
}
```

A. Compare o1 == o2
B. Override only hashCode().
C. Override both equals() and hashCode() based on orderId.
D. Make orderId static.

Correct answer: C. Override both equals() and hashCode() based on orderId.
Explanation: Override equals() for logical equality and hashCode() consistently, which is required for hash-based collections.

Q25. What is the output?

```
String s1 = "Ja" + "va";
String s2 = "Java";
System.out.println(s1 == s2);
```

A. true
B. false
C. Compilation error
D. Runtime Error

Correct answer: A. true
Explanation: The compiler folds the constant expression to "Java", so both refer to the same pooled String.

Q26. What is the output?

```
String a = "Java";
String b = "Ja";
String c = b + "va";
System.out.println(a == c);
System.out.println(a == c.intern());
```

A. true, true
B. false, true
C. false, false
D. true, false

Correct answer: B. false, true
Explanation: b is not final, so b + "va" is built at runtime on the heap (a ≠ c). c.intern() returns the pooled "Java" (a == intern is true).

Q27. What is the output?

```
String s1 = new String("Spring");
String s2 = s1.intern();
System.out.println(s1 == s2);
System.out.println(s2 == "Spring");
```

A. true, true
B. false, false
C. false, true
D. true, false

Correct answer: C. false, true
Explanation: s1 is a heap copy; s2 is the pooled instance, so s1 == s2 is false. The literal "Spring" is the same pooled instance, so s2 == "Spring" is true.

Q28. What is the output?

```
final String x = "Ja";
String s1 = x + "va";
String s2 = "Java";
System.out.println(s1 == s2);
```

A. true
B. false
C. Compilation error
D. Depends on garbage collection

Correct answer: A. true
Explanation: x is a final constant variable, so x + "va" is a compile-time constant interned as "Java". Result: true.

Q29. Which approach is generally best for repeated string concatenation inside a loop?

A. Use result.intern() on every iteration.
B. Use new String(result) on every iteration.
C. Use final String result.
D. Use StringBuilder and append each value.

Correct answer: D. Use StringBuilder and append each value.
Explanation: Strings are immutable, so + in a loop creates many temporary objects. StringBuilder appends in place.

Q30. What is the output?

```
String s = new String("Ja") + new String("va");
s.intern();
String s1 = "Java";
System.out.println(s == s1);
```

A. true
B. false
C. Compilation error
D. Runtime exception

Correct answer: A. true
Explanation: "Java" isn't in the pool yet when s.intern() runs, so intern() puts s itself into the pool. The later literal "Java" resolves to that same object, so s == s1 is true (Java 7+).

Q31. What is the output?

```
public class Test {
    public static void main(String[] args) {
        String s1 = "Java";
        String s2 = new String("Java");
        s2.intern();
        System.out.println(s1 == s2);
    }
}
```

A. true
B. false
C. Compilation error
D. Runtime exception

Correct answer: B. false
Explanation: s2.intern() returns the pooled String but the result is discarded, so s2 still points to the heap object. s1 == s2 is false.